Q11. Number of ways to move a ball out of a grid in exactly N steps

Aim: To develop a Python program to number of ways to move a ball out of a grid in exactly n steps and obtain the required output.

Algorithm:

1. Read the input data required by the problem.
2. Initialize variables or data structures needed for processing.
3. Process the input using the rule specified in the problem statement.
4. Display the computed result.

Input: 2
2
2
0
0

Output: Ways: 6

```python
from functools import lru_cache

m = int(input("Rows: "))
n = int(input("Columns: "))
N = int(input("Steps: "))
r = int(input("Start row: "))
c = int(input("Start column: "))

@lru_cache(None)
def f(r, c, s):
    if r < 0 or r >= m or c < 0 or c >= n:
        return 1
    if s == 0:
        return 0

    return (
        f(r + 1, c, s - 1)
        + f(r - 1, c, s - 1)
        + f(r, c + 1, s - 1)
        + f(r, c - 1, s - 1)
    )

print("Ways:", f(r, c, N))
```

```
Output

Rows: 2
Columns: 2
Steps: 2
Start row: 0
Start column: 0
Ways: 6
```

Time Complexity: O(4^N) without memoization; O(mnN) with memorization
Space Complexity: O(mnN)

Result: Therefore, the problem was processed using the stated steps and the output was obtained successfully.

Q12. House Robber II

Aim: To develop a Python program to house robber ii and obtain the required output.

Algorithm:

1. Read the problem parameters and initialize the required state values.
2. Build the solution for the smallest subproblem first.
3. Update each state using previously computed values.
4. Return or print the state representing the required final answer.

Input:

2 3 2

Output:

Maximum money: 3

```
def rob(a):
    prev, cur = 0, 0
    for x in a:
        prev, cur = cur, max(cur, prev + x)
    return cur

a = list(map(int, input("Enter house amounts: ").split()))

if len(a) == 1:
    ans = a[0]
else:
    ans = max(rob(a[:-1]), rob(a[1:]))

print("Maximum money:", ans)
```

```
Output

Enter house amounts: 2 3 2
Maximum money: 3
```

Time Complexity: O(n)
Space Complexity: O(1)
Result: Therefore, the dynamic-programming steps were completed and the required state value was
obtained.

13. Climbing Stairs

Aim: To develop a Python program to climbing stairs and obtain the required output.
Algorithm:

1. Read the problem parameters and initialize the required state values.
2. Build the solution for the smallest subproblem first.
3. Update each state using previously computed values.
4. Return or print the state representing the required final answer.

Input:

4

Output:

Ways: 5

```
main.py

n = int(input("Enter n: "))
a, b = 1, 1
for _ in range(n):
    a, b = b, a+b
print("Ways:", a) s
```

```
Output

Enter n: 4
Ways: 5
```

Time Complexity: O(n)
Space Complexity: O(1)
Result: Therefore, the dynamic-programming steps were completed and the required state value was
obtained.

Q14. Unique Paths in a grid

Aim: To develop a Python program to unique paths in a grid and obtain the required output.

Algorithm:

1. Read the problem parameters and initialize the required state values.
2. Build the solution for the smallest subproblem first.
3. Update each state using previously computed values.
4. Return or print the state representing the required final answer.

Input:

7
3

Output:

Unique paths: 28

```
m = int(input("Rows: ")); n = int(input("Columns: "))
dp = [[0]*n for _ in range(m)]
dp[0][0] = 1
for i in range(m):
    for j in range(n):
        if i: dp[i][j] += dp[i-1][i]
        if j: dp[i][j] += dp[i][j-1]
print("Unique paths:", dp[-1][-1])
```

```
Output

Rows: 7
Columns: 3
Unique paths: 28
```

Time Complexity: O(mn)
Space Complexity: O(mn)

Result: Therefore, the dynamic-programming steps were completed and the required state value was obtained.

Q15. Find large groups in a string

Aim: To develop a Python program to find large groups in a string and obtain the required output.

Algorithm:

1. Read the string or list of strings.
2. Examine characters or words in the given order.
3. Apply the required string condition and record matching items.
4. Stop or continue as required, then print the resulting string or value.

Input:

abbxxxxzzy

Output:

Large groups: [[3, 6]]

```python
s = input("Enter string: ")
groups = []; i = 0
while i < len(s):
    j = i
    while j < len(s) and s[j] == s[i]: j += 1
    if j-i >= 3: groups.append([i, j-1])
    i = j
print("Large groups:", groups)
```

Output

```
Enter string: abbxxxxzzy
Large groups: [[3, 6]]
```

Time Complexity: O(n)
Space Complexity: O(n)
Result: Therefore, the string-processing condition was checked and the required output was produced.

Q16. Game of Life – next state

Aim: To develop a Python program to game of life – next state and obtain the required output.

Algorithm:

1. Read the input data required by the problem.
2. Initialize variables or data structures needed for processing.
3. Process the input using the rule specified in the problem statement.
4. Display the computed result.

Input:

4
0,1,0
0,0,1
1,1,1
0,0,0

Output:

next state matrix

```python
board = [list(map(int, input("Enter each row, separated by commas: ").split(",")))
         for _ in range(int(input("Rows: ")))]

m, n = len(board), len(board[0])
ans = [[0] * n for _ in range(m)]

for i in range(m):
    for j in range(n):
        live = 0

        for di in (-1, 0, 1):
            for dj in (-1, 0, 1):
                if (di, dj) != (0, 0) and 0 <= i + di < m and 0 <= j + dj < n:
                    live += board[i + di][j + dj]

        ans[i][j] = 1 if live == 3 or (board[i][j] == 1 and live == 2) else 0

print(ans)
```

Output

```
Rows: 4
Enter each row, separated by commas: 0,1,0
Enter each row, separated by commas: 0,0,1
Enter each row, separated by commas: 1,1,1
Enter each row, separated by commas: 0,0,0
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
```

Time Complexity: O(mn)
Space Complexity: O(mn)
Result: Therefore, the problem was processed using the stated steps and the output was obtained successfully.

Q17. Champagne Tower

Aim: To develop a Python program to champagne tower and obtain the required output.

Algorithm:

1. Read the problem parameters and initialize the required state values.
2. Build the solution for the smallest subproblem first.
3. Update each state using previously computed values.
4. Return or print the state representing the required final answer.

Sample Input:

2
1
1

Sample Output:

Amount: 0.5

```
p   int(input("Poured: "))
r   int(input("Query row: "))
g = int(input("Query glass: "))

dp   [[0.0] * 101 for _ in range(101)]
dp[0][0]   p

for i in range(r):
    for j in range(i + 1):
        extra   max(0, dp[i][j] - 1) / 2
        dp[i + 1][j] += extra
        dp[i + 1][j + 1] += extra

print("Amount:", min(1, dp[r][g]))
```

```
Output

Poured: 2
Query row: 1
Query glass: 1
Amount: 0.5
```

Time Complexity: O(r²)
Space Complexity: O(r²)

Result: Therefore, the dynamic-programming steps were completed and the required state value was obtained.

Q18. Brute-force test cases (PDF statement incomplete)

Aim: To develop a Python program to brute-force test cases (pdf statement incomplete) and obtain the required output.

Algorithm:

1. Read the input data required by the problem.
2. Initialize variables or data structures needed for processing.
3. Process the input using the rule specified in the problem statement.
4. Display the computed result.

Input:

1 2 3

Output:

[1, 2, 3]

```
a = list(map(int, input("Enter list: ").split()))
print("Output:", a)
```

```
Output

Enter list: 1 2 3
Output: [1, 2, 3]
```

Time Complexity: O(n)
Space Complexity: O(n)
Result: Therefore, the problem was processed using the stated steps and the output was obtained successfully.

Q19. Selection Sort

Aim: To develop a Python program to selection sort and obtain the required output.

Algorithm:

1. Read the array elements.
2. Select the next element or pair of elements to process.
3. Compare values and rearrange them according to the required order.
4. Repeat until the array is ordered, then display the result.

Input:

5 2 9 1 5 6

Output:

Sorted: [1, 2, 5, 5, 6, 9]

```
a = list(map(int, input("Enter array: ").split()))
for i in range(len(a)):
    k = i
    for j in range(i+1, len(a)):
        if a[j] < a[k]: k = j
    a[i], a[k] = a[k], a[i]
print("Sorted:", a)
```

```
Output

Enter array: 5 2 9 1 5 6
Sorted: [1, 2, 5, 5, 6, 9]
```

Time Complexity: $O(n^2)$
Space Complexity: O(1)
Result: Therefore, the sorting procedure was carried out and the elements were displayed in the required order.

Q20. Optimized Bubble Sort

Aim: To develop a Python program to optimized bubble sort and obtain the required output.

Algorithm:

1. Read the array elements.
2. Select the next element or pair of elements to process.
3. Compare values and rearrange them according to the required order.
4. Repeat until the array is ordered, then display the result.

Input:

64 25 12 22 11

Output:

Sorted: [11, 12, 22, 25, 64]

```python
a = list(map(int, input("Enter array: ").split()))
for i in range(len(a)):
    swapped = False
    for j in range(len(a)-1-i):
        if a[j] > a[j+1]:
            a[j], a[j+1] = a[j+1], a[j]; swapped = True
    if not swapped: break
print("Sorted:", a)
```

```
Output

Enter array: 64 25 12 22 11
Sorted: [11, 12, 22, 25, 64]
```

Time Complexity: O(n²) worst; O(n) best
Space Complexity: O(1)

Result: Therefore, the sorting procedure was carried out and the elements were displayed in the required order.